UVM Testbench

AXI-Lite Slave IP with Memory

ECE-593: Fundamentals of Pre-Si Validation
Portland State University
Team: Patrick, Pratibha

1. Overview

This document describes the SystemVerilog UVM 1.2 verification environment for the AXI4-Lite Slave IP. The environment is a port of the MS2/MS3 class-based testbench onto standard UVM base classes: the generator and mailbox pair is replaced by a `uvm_sequencer` driving a library of `uvm_sequence` classes; the driver, monitor, and scoreboard extend `uvm_driver`, `uvm_monitor`, and `uvm_scoreboard`; and the environment extends `uvm_env`. UVM infrastructure replaces the hand-coded glue from MS2/MS3 — `uvm_config_db` carries the virtual interface through the hierarchy, `uvm_phase` objections control simulation end, and TLM analysis ports replace the `mon2scb` mailboxes. All testbench prints route through the UVM reporting subsystem.

2. Architecture

The environment is organized into eleven source files under `AXI_Lite_MS4_UVM/tb/`. Every component and sequence is parameterized on `DATA_WIDTH`, `ADDR_WIDTH`, and `MEM_DEPTH` and registered with `uvm_object_param_utils` or `uvm_component_param_utils` accordingly.

File	Class / Module	Responsibility
axil_if.sv	axil_if	AXI4-Lite interface (carried over from MS2): bundles all signals, defines master/monitor clocking blocks and a reset task.
axil_seq_item.sv	axil_seq_item	UVM sequence item: data object holding all fields for a single read, write, or concurrent read+write (TXN_BOTH) operation. Field macros enabled for automated print/copy/compare.
axil_sequencer.sv	axil_sequencer	Parameterized typedef wrapper around uvm_sequencer. Manages the flow of sequence items from running sequences to the driver.
axil_driver.sv	axil_driver	AXI-Lite master BFM. Pulls items via seq_item_port, drives all five AXI channels through the virtual interface, then signals item_done.
axil_monitor.sv	axil_monitor	Passive observer. Five concurrent threads sample the bus, reconstruct write and read transactions, and publish them on a

File	Class / Module	Responsibility
		uvm_analysis_port. Hosts cycle-level functional coverage.
axil_agent.sv	axil_agent	Active container for driver, sequencer, and monitor. Connects driver to sequencer and forwards the monitor's analysis port up to the environment.
axil_scoreboard.sv	axil_scoreboard	Maintains a shadow memory model, checks every observed transaction via analysis_imp, and hosts transaction-level functional coverage.
axil_env.sv	axil_env	Top-level UVM container: instantiates agent and scoreboard, wires the monitor's analysis port to the scoreboard in connect_phase.
axil_sequences.sv	axil_base_seq + 5	Sequence library: base sequence with shared helpers, plus five derived sequences (random, wr_rd, byte_strobe, concurrent, addr_oor).
axil_tests.sv	axil_test	Test class: builds the environment, prints UVM topology, raises objection, runs all five sequences in turn, drops objection.
axil_tb_top.sv	axil_tb_top	Static top module: clock generation, interface and DUT instantiation, uvm_config_db setup, reset application, and run_test() invocation.

3. Component Detail

3.1 Sequence Item (axil_seq_item)

The sequence item extends `uvm_sequence_item` and carries the same payload fields as the MS2 transaction class: a `txn_mode_e` enumeration (TXN_WRITE / TXN_READ / TXN_BOTH), the addressing and write payload fields (`addr`, `wdata`, `wstrb`, `prot`), and the response fields populated by the monitor (`rdata`, `bresp`, `rresp`). All fields are registered with `uvm_field_*` macros inside `uvm_object_param_utils_begin` / `uvm_object_utils_end`, with `UVM_HEX` formatting on address and data fields and `UVM_BIN` on strobe, protection, and response fields. This enables automated `print()`, `copy()`, `compare()`, `pack()`, and `record()` operations. Three constraints carry over unchanged from MS2: `c_addr_align` (word-aligned), `c_wstrb_nonzero` (non-zero strobe on writes), and `c_mode_random` (random items restricted to WRITE/READ; BOTH unlocked only in directed sequences via `constraint_mode(0)`). A custom `convert2string()` overrides the auto-generated print formatting where a compact one-line representation is preferred for transaction-level log messages.

3.2 Sequencer (axil_sequencer)

The sequencer is a thin parameterized typedef wrapper around `uvm_sequencer #(axil_seq_item)`. No additional functionality is added — the class exists to complete the standard UVM component set and to allow future extension (custom arbitration, configuration handle, p_sequencer access from sequences).

3.3 Driver (axil_driver)

The driver extends `uvm_driver` and pulls items from the sequencer via `seq_item_port.get_next_item()`, dispatches to the appropriate drive task based on the item's mode, and signals completion with `item_done()`. All MS2 BFM tasks carry over unchanged: `drive_write` forks parallel AW and W channel handshakes then waits for B response; `drive_read` drives AR then asserts RREADY; `drive_both` aligns AW, W, and AR to the same clock edge using the maximum of their individual random delays, waits for all three ready signals simultaneously, then handles B and R responses in order. The five per-channel delay knobs (`max_aw_delay`, `max_w_delay`, `max_ar_delay`, `max_b_delay`, `max_r_delay`) carry over from MS2 to allow back-pressure stress testing without a separate test class. The virtual interface is retrieved from `uvm_config_db` in `build_phase` and the driver loops on the sequencer in `run_phase`.

3.4 Monitor (axil_monitor)

The monitor extends `uvm_monitor` and runs five concurrent threads inside a `fork/join_none` block in `run_phase`:

- `monitor_aw()`: captures AW-channel handshakes into an internal aw_mbx.
- `monitor_w()`: captures W-channel handshakes into an internal w_mbx.
- `merge_write()`: pairs AW and W objects, waits for the B handshake to obtain bresp, then writes a merged transaction to `monitor_port` (`uvm_analysis_port`).
- `monitor_read()`: captures the AR handshake then the R handshake and writes the complete read transaction to `monitor_port`.
- `cycle_coverage()`: a continuous cycle-level tracker that samples the six cycle-level covergroups every clock edge (described in Section 4).

All five threads, the merging logic, and every covergroup sample carry over unchanged from the MS2 monitor. The only structural change is the replacement of the two `mon2scb` mailboxes with a single forwarded `uvm_analysis_port`.

3.5 Agent (axil_agent)

The agent extends `uvm_agent` and is a new file split from the MS2 environment. It owns three children: the sequencer, the driver, and the monitor. In `connect_phase` it wires the driver's `seq_item_port` to the sequencer's `seq_item_export`, and forwards the monitor's `monitor_port` up to the environment via its own `monitor_port` so the scoreboard can subscribe without reaching into the agent's internals.

3.6 Scoreboard (axil_scoreboard)

The scoreboard extends `uvm_scoreboard` and maintains the same shadow memory model as MS2 — a `[0:MEM_DEPTH-1]` array of `DATA_WIDTH` words, optionally preloaded from a hex file via `uvm_config_db`. It exposes a `uvm_analysis_imp` named `analysis_imp`; its non-blocking `write()` callback pushes incoming transactions onto a queue, which `run_phase` drains and dispatches to `check_writes()` or `check_reads()` based on the item's mode. On every write it applies the byte-strobe mask to update the shadow at the word address computed from `addr`; on every read it compares `rdata` against the shadow value and flags any mismatch with `uvm_error`. BRESP and RRESP are independently checked for OKAY (2'b00); SLVERR (2'b10) is allowed only for out-of-range addresses, and any other code is reported as an error. Statistics (writes_checked, reads_checked, errors) and per-covergroup percentages are reported automatically by `report_phase`.

3.7 Environment (axil_env)

The env class extends `uvm_env` and is a thin container that instantiates the agent and the scoreboard in `build_phase`, then wires `agent.monitor_port.connect(scoreboard.analysis_imp)` in `connect_phase`. The MS2 lifecycle methods (`init`, `run`, `drain`, `stop`, `report`) are removed — UVM phase objections in the test control simulation termination, and `report_phase` is invoked automatically by the UVM phasing mechanism.

3.8 Sequence Library (axil_sequences)

All sequences extend `axil_base_seq`, which itself extends `uvm_sequence #(axil_seq_item)` and provides three protected helpers — `send_write()`, `send_read()`, and `send_both()` — built on `start_item` / `finish_item` with inline randomize-with constraints, plus a `rand_mem_addr()` helper for in-range word addresses.

Sequence	Type	Purpose
axil_random_seq	Constrained-random	Sends 50 word-aligned WRITE/READ transactions across the in-range memory window. Address constrained via the rand_mem_addr() helper to 0 <= word_idx < MEM_DEPTH.
axil_wr_rd_seq	Directed	Writes the pattern 0xDEAD_000N to eight word-aligned addresses then reads each back to confirm data integrity.
axil_byte_strobe_seq	Directed	Builds the value 0x12345678 one byte at a time using single-byte WSTRBs over the same address, plus a sweep across all non-zero strobe values (1..15) to exercise WSTRB byte-lane masking.
axil_concurrent_seq	Directed	Issues TXN_BOTH transactions that drive AW + W + AR simultaneously on the same clock edge. Exercises the DUT's write-before-read forwarding path.

Sequence	Type	Purpose
axil_addr_oor_seq	Directed	Drives a write and a read at MEM_DEPTH*(DATA_WIDTH/8)+4 to confirm BRESP/RRESP = SLVERR on out-of-range accesses.

3.9 Test Class (axil_test)

A single unified test class extends `uvm_test`. In `build_phase` it creates the environment and instantiates all five sequence objects. In `start_of_simulation_phase` it sets up UVM logging — opens `axil_uvm.log` and routes every `uvm_*` message at this component and below to both the simulator transcript and the log file via the report-action API (see Section 5.1 for details). In `end_of_elaboration_phase` it calls `uvm_top.print_topology()` so the UVM component tree is dumped to the transcript before simulation starts. In `run_phase` it raises an objection, starts each sequence in turn on `env.agent.sequencer` via `.start()`, then drops the objection to end the simulation. The log file is closed cleanly in `final_phase`. Per-sequence start markers are emitted with `uvm_info` so the transcript clearly delineates the five test scenarios.

3.10 Testbench Top (axil_tb_top)

The static top module instantiates the clock generator (100 MHz, 10 ns period), the AXI-Lite interface, and the DUT. The MS2 sequential test execution block is removed — `run_test("axil_test")` dispatches the unified test. The virtual interface and the MEM_INIT path are pushed once into `uvm_config_db` so all components can retrieve them through the hierarchy. Reset is applied via `axil_bus.do_reset()` once before `run_test()` so the DUT is in a known state when the first sequence starts; tests no longer call reset themselves.

4. Functional Coverage

Functional coverage is unchanged from MS3. Coverage is split between the monitor (cycle-level covergroups, sampled every clock edge by a dedicated tracker thread) and the scoreboard (transaction-level covergroups, sampled after each scoreboard check). Both report per-group and average coverage in `report_phase`.

ID	Covergroup	Owner	What is Measured
FV-001	cg_reset	Monitor	Reset asserted and de-asserted states.
FV-002/ 003	cg_aw_w_hs	Monitor	AW-first, W-first, and concurrent AW+W handshake orderings.
FV-005	cg_bresp	Scoreboard	BRESP codes: OKAY and SLVERR.
FV-006	cg_wstrb	Scoreboard	All 15 valid WSTRB byte-lane patterns (single, two, three, and full byte writes).
FV-007/ 010	cg_oor	Scoreboard	Cross of in-range/out-of-range address vs. write/read operation type and response code.

ID	Covergroup	Owner	What is Measured
FV-008	cg_ar_hs	Monitor	AR-to-R channel latency binned: 1, 2, 3, 4-7, 8-15, and 16+ cycles.
FV-009	cg_rresp	Scoreboard	RRESP codes: OKAY and SLVERR.
FV-011	cg_fwd	Monitor	Same-cycle AW and AR handshakes to the same word address (write-before-read forwarding scenario).
FV-012	cg_addr	Scoreboard	Word address distribution across four equal quarters of the memory map, plus out-of-range.
FV-013	cg_bp_b / cg_bp_r	Monitor	BREADY and RREADY back-pressure stall duration binned: 0, 1-3, 4-7, 8-15, and 16+ cycles.
FV-014	cg_b2b	Scoreboard	Back-to-back transaction ordering: write-write, write-read, read-write, and read-read pairs.

5. Messaging and Logging

All testbench prints route through the UVM reporting subsystem rather than `$display`. The four standard severity macros are used throughout with consistent ID strings so output can be filtered at run time. Verbosity levels are graded so the default run produces a readable transcript while a higher verbosity reveals every component-creation and per-transaction event.

Macro	Severity	Used For
`uvm_info`	INFO	Per-transaction prints, lifecycle events, end-of-test summaries, coverage reports.
`uvm_warning`	WARNING	Recoverable anomalies (e.g., shadow-memory write outside MEM_DEPTH).
`uvm_error`	ERROR	Scoreboard mismatches, unexpected BRESP/RRESP encodings. Counted toward the final pass/fail decision.
`uvm_fatal`	FATAL	Configuration failures that prevent the testbench from running (e.g., virtual interface not found in uvm_config_db). Terminates simulation immediately.

Eight ID strings identify the originating component at a glance: `DRV` (driver), `MON` (monitor), `SCB` (scoreboard), `ENV` (environment), `AGT` (agent), `SEQ` (sequencer/sequences), `TEST` (top-level test), and `CFG` (config DB failures, fatal). End-of-test summaries (pass/fail counts, per-covergroup percentages, RESULT line) are emitted at `UVM_NONE` so they always appear in the transcript. Per-transaction driver and monitor prints are at `UVM_MEDIUM` (the default). Component-creation traces are at `UVM_HIGH` for deep debugging.

5.1 UVM Logging

In addition to the simulator transcript, every `uvm_*` message is captured to a persistent log file `axil_uvm.log` via the UVM report-action API. The setup lives in

`axil_test::start_of_simulation_phase` so it runs once after build and applies hierarchically to every child component (env, agent, driver, monitor, scoreboard):

```
virtual function void start_of_simulation_phase(uvm_phase phase);
  super.start_of_simulation_phase(phase);
  log_fd = $fopen("axil_uvm.log", "w");
  if (log_fd == 0)
    `uvm_fatal("LOG", "Failed to open axil_uvm.log for writing")
  set_report_default_file_hier(log_fd);
  set_report_severity_action_hier(UVM_INFO,    UVM_DISPLAY | UVM_LOG);
  set_report_severity_action_hier(UVM_WARNING, UVM_DISPLAY | UVM_LOG);
  set_report_severity_action_hier(UVM_ERROR,   UVM_DISPLAY | UVM_LOG | UVM_COUNT);
  set_report_severity_action_hier(UVM_FATAL,   UVM_DISPLAY | UVM_LOG | UVM_EXIT);
endfunction
```

Each severity is routed to both `UVM_DISPLAY` (the transcript) and `UVM_LOG` (the file). `UVM_ERROR` additionally carries `UVM_COUNT` so the simulation terminates after the configured error limit; `UVM_FATAL` carries `UVM_EXIT` so the simulation stops immediately. The log file is closed cleanly in `final_phase`. The `+UVM_LOG_FILE=<name>` plusarg can override the default filename at run time without rebuilding.

Compile and run on Synopsys VCS:

```
vcs -sverilog -ntb_opts uvm-1.2 -full64 \
    +incdir+. axil_tb_top.sv s_axil_top.v \
    -timescale=1ns/1ps
./simv                                  # default: log to axil_uvm.log
./simv +UVM_VERBOSITY=UVM_HIGH          # full traces
./simv +UVM_LOG_FILE=my_run.log         # custom log file
```